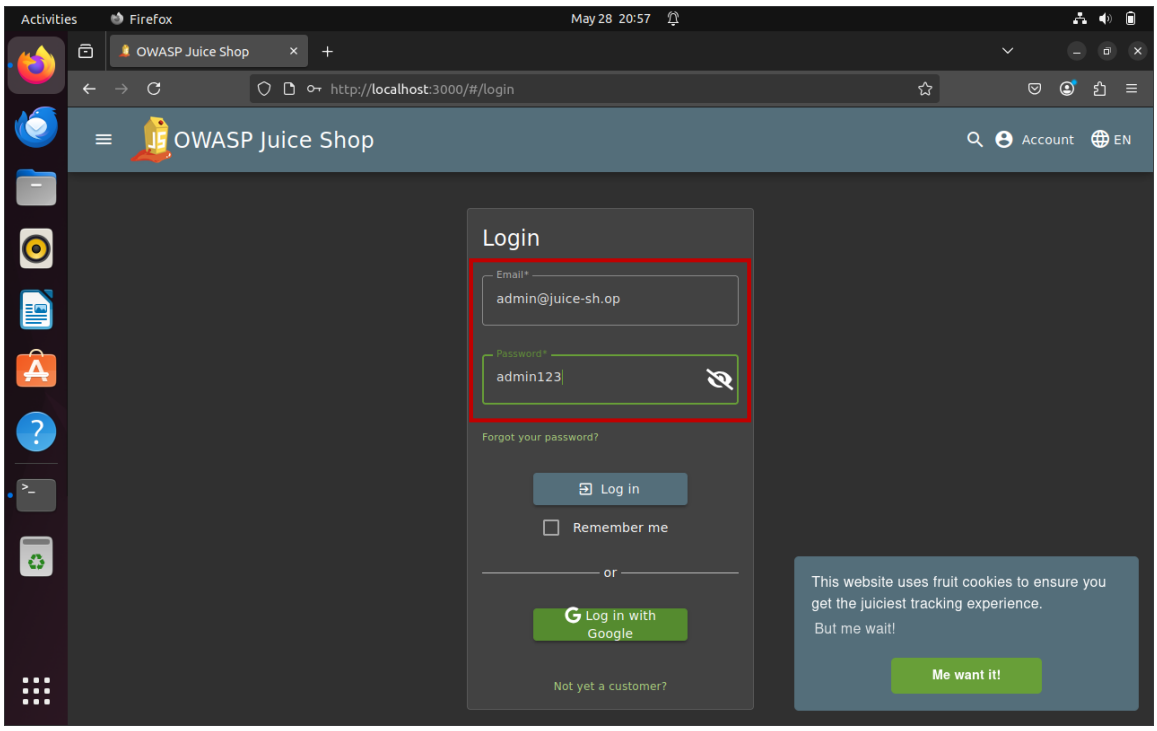
Jan Harvey C. Javier **05-28-2025**

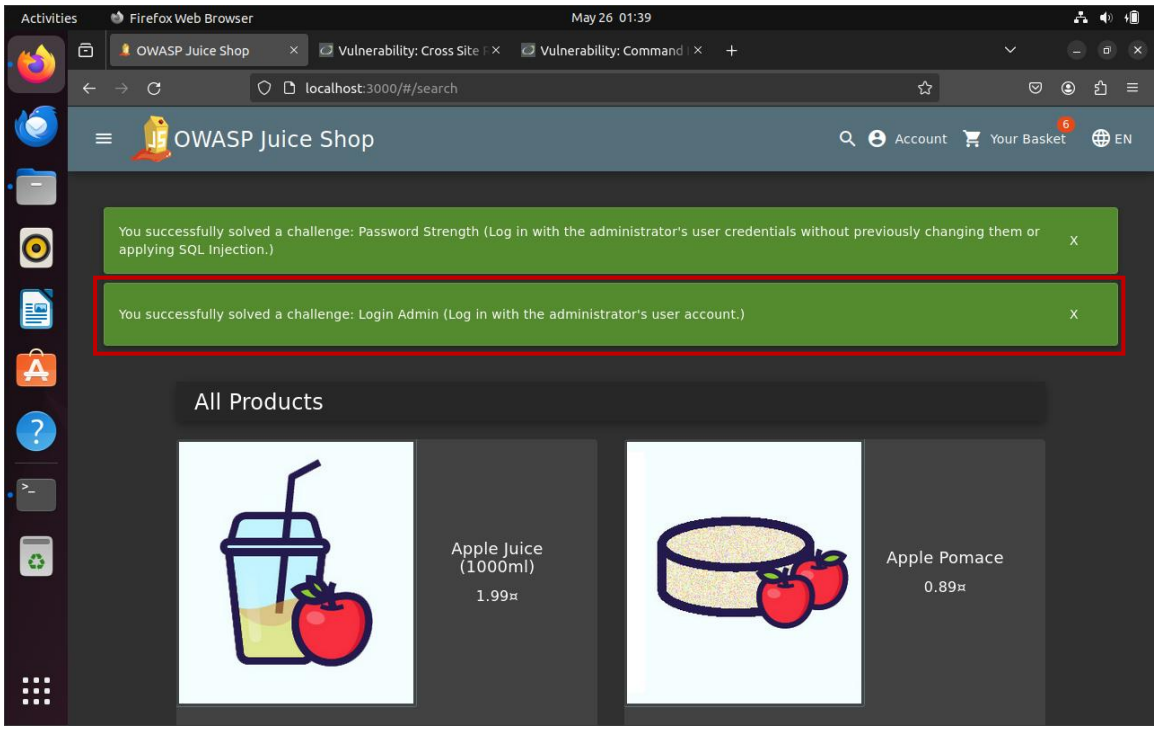
BSIT-3B **Sir Benedick Sarmiento**

Analyzing and Fixing Common Security Flaws in Software Design

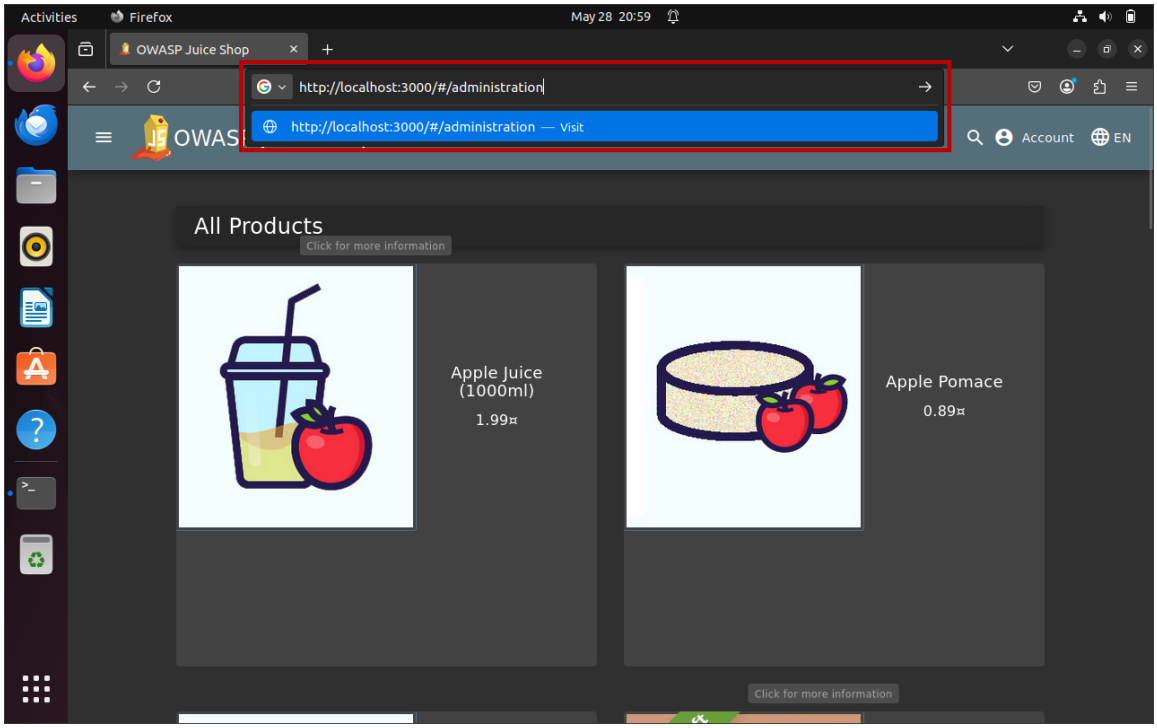
Part 1: Vulnerability Exploration → Identifying Security Flaws

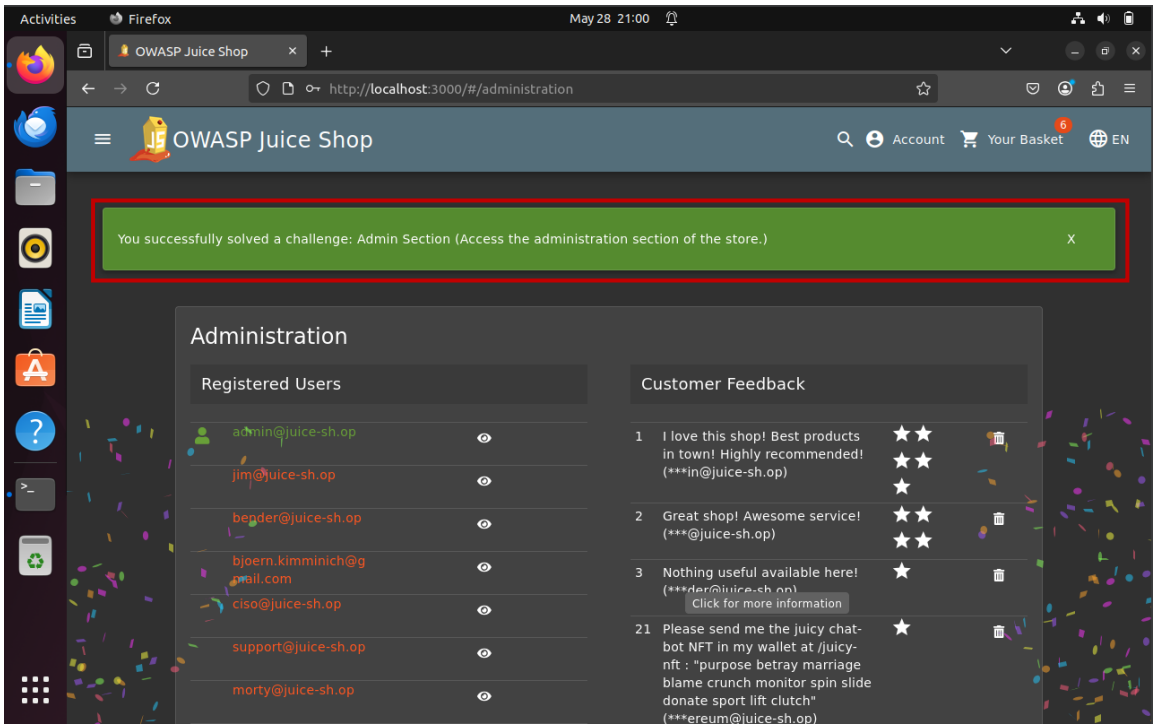
- **Poorly Designed Authentication Mechanism**
  - Logging in using default credentials – indicating a flawed authentication check or weak default passwords.



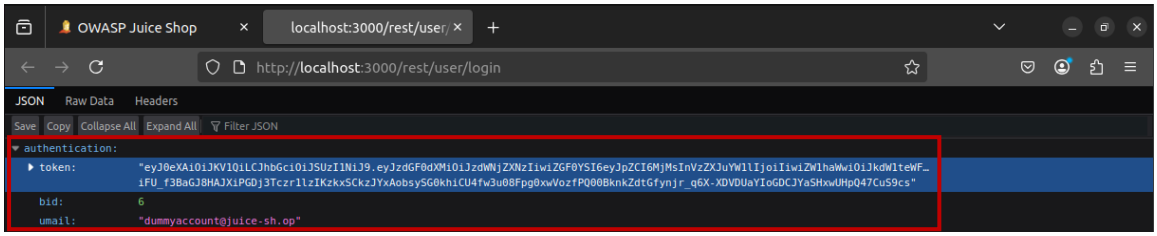


- **Inadequate Access Control**
  - Accessing the admin panel without proper authorization – this shows missing role-based access checks on the backend.



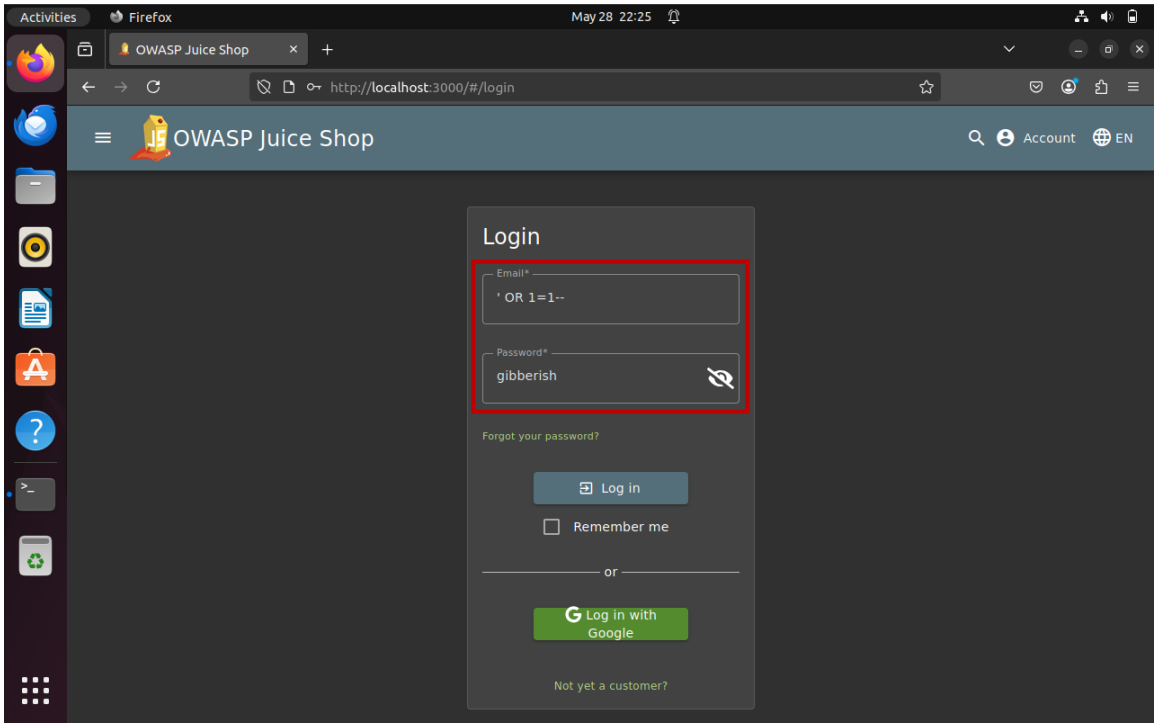


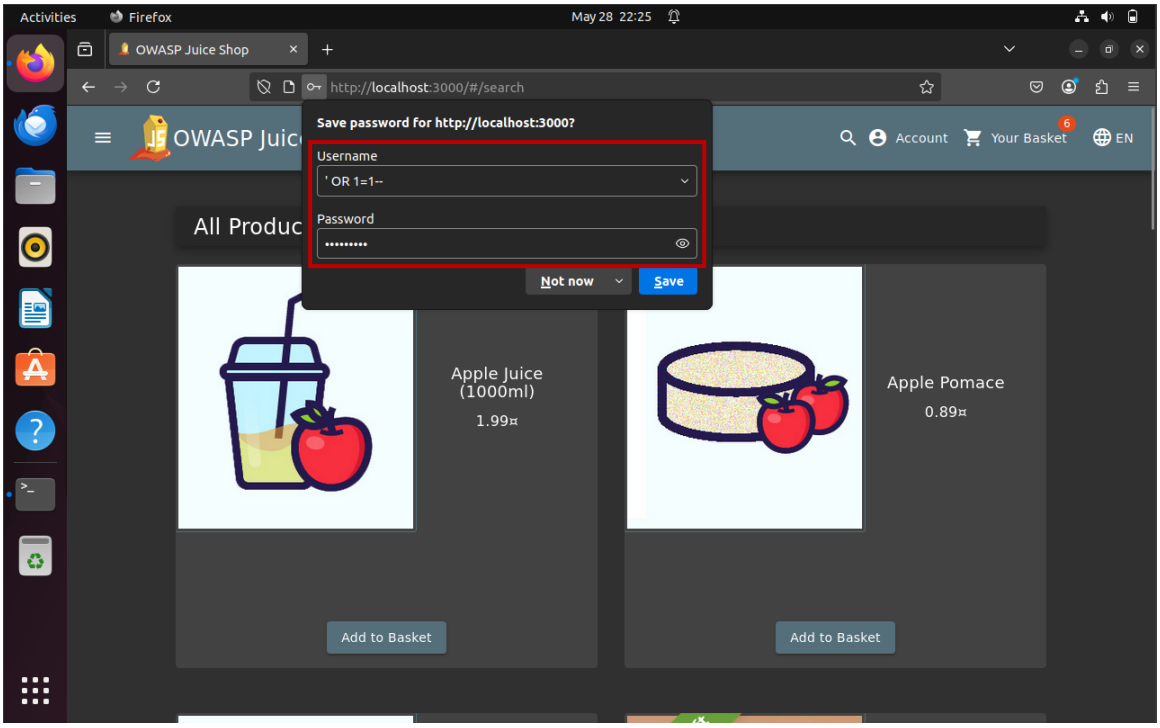
- **Unencrypted Sensitive Data**
  - Finding .json file in Developer Tools – this shows plain-text credentials like tokens and emails in application files that should be encrypted or at least stored securely.



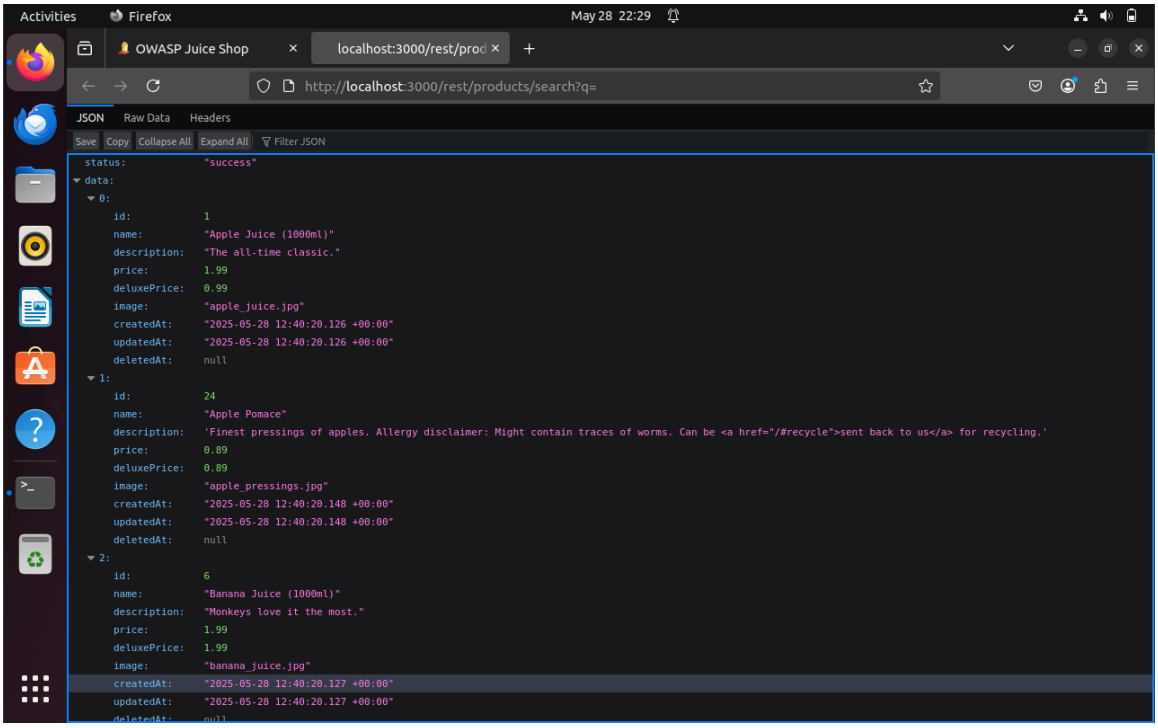
- **Network-related vulnerabilities**
  - HTTP instead of HTTPS and no warning icon or no icon at all at the left side of address bar – this shows insecure webpage.

- **Inadequate Input Validation**
  - Logging in using an SQL Injection - showing backend SQL queries are not properly parameterized.

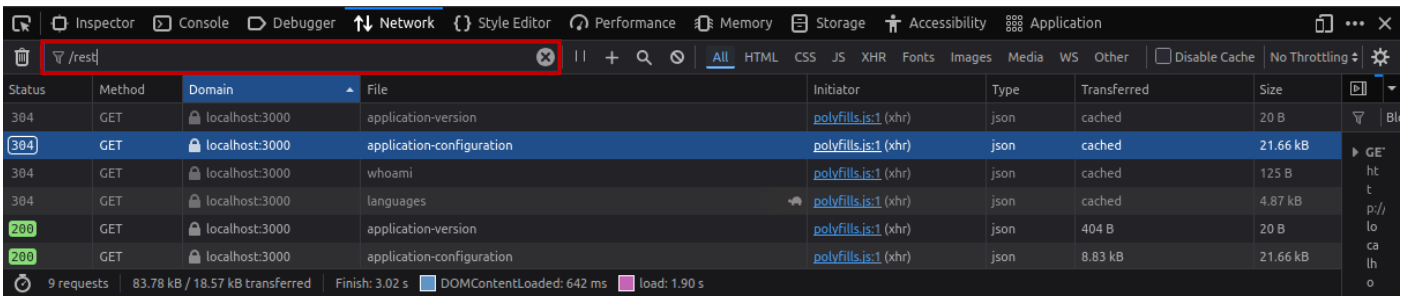




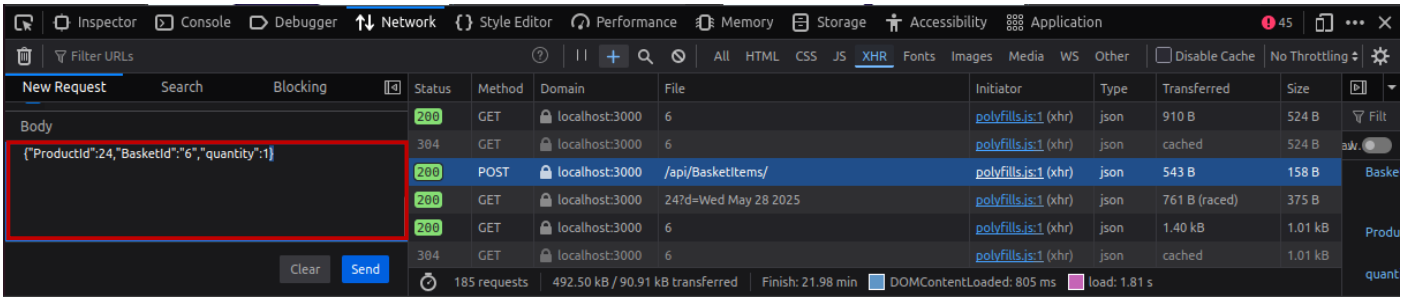
- **Error messages leaking technical details**
  - Triggering an error by accessing a non-existing product – this returns stack trace or database structure.



- **Weak Architectural Decisions (No API Gateway or Microservice Isolation)**
    - Searching '/rest' on domain in Developer Tools - showing all APIs are exposed directly and lack service boundaries.



- **Lack of logging/monitoring mechanisms**
    - In Firefox Developer Tools, I was able to modify the API request undetected – this shows no alert or rejection occur in terminal logs or backend files.



- **Absence of Secure Development Practices (hardcoded secrets, missing tests)**
    - OWASP Juice Shop's hardcoded secrets (token, email, password) are hidden by the author/developer.

Part 2: Design and Code Fixes

Issue 1: Poorly Designed Authentication Mechanism

Describe why the current design is insecure

- Juice Shop allows weak passwords (like "123456") and doesn't enforce multi-factor authentication. Tokens are stored in local storage, which is vulnerable to cross-site scripting attacks.

Propose a secure design alternative

- Enforce password complexity
- Use multi-factor authentication
- Store tokens in secure cookies

Code-level or architectural improvements

```
const passwordSchema = new passwordValidator();
passwordSchema
  .is().min(8)
  .has().uppercase()
  .has().lowercase()
  .has().digits()
  .has().symbols();

if (!passwordSchema.validate(req.body.password)) {
  return res.status(400).json({ error: 'Weak password' });
}
```

Issue 2: Inadequate Access Control

Describe why the current design is insecure

- APIs like /api/BasketItems/ let any authenticated user modify another user's cart by tampering with the BasketId.

Propose a secure design alternative

- Enforce object-level access control on the server.
- Validate ownership of the basket before modifying it.

Code-level or architectural improvements

```
if (req.user.id !== basket.UserId) {
  return res.status(403).json({ error: 'Unauthorized access' });
}
```

Issue 3: Error messages leaking technical details

Describe why the current design is insecure

- Error responses include stack traces or internal details, which can help attackers craft targeted exploits.

Propose a secure design alternative

- Show user-friendly errors to users.
- Log detailed errors only on the server.

Code-level or architectural improvements

```
app.use((err, req, res, next) => {
  console.error(err.stack); // Internal logging
  res.status(500).json({ error: 'Something went wrong. Please try again later.'});
```

Part 3: Reflection and Submission

1. **Which types of flaws did you find most concerning and why?**

    - The most concerning type of flaw is Inadeqaute Access Control because it bypasses all permission boundaries, potentially giving attackers access to sensitive user data or admin functionality and often leads to privilege escalation, data tampering, and account takeover. These kind of flaw is also commonly exploit and hard to detect.

2. **What secure design principles could have prevented these flaws?**

    - Principle of Least Privilege
        - Users and processes should have the minimum level of access necessary to perform their tasks.
    - Fail Securely
        - If an access check fails or is missing, deny access by default.
    - Complete Mediation
        - Every request to a resource must be checked for proper authorization
    - Secure by Design
        - Security should be a core consideration from the beginning

3. **How can development teams integrate secure practices into their workflow?**

    - Development teams can integrate secure practices by adopting coding standards, perform code reviews with security focus including security checkpoints in peer reviews, conducting penetration or security testing, and lastly conducting training and awareness by educating developers with secure design principles.